CSL7110 Assignment 2
MinHashing and Locality Sensitive Hashing

Name: Kunal Mishra
Roll Number: M25CSA036

Course Name: Machine Learning with Big Data (CSL7110)

GitHub Repository:
Repo Link

1. K-Grams and Jaccard Similarity

Generate character 2-grams, character 3-grams, and word 2-grams for the given documents and compute the Jaccard similarity between all document pairs using these representations.

Answer:

In this section, similarity between four documents was analyzed using **k-gram based representations** and **Jaccard similarity**.

First, character-level k-grams were generated by sliding a window of size *k* across each document. Both **character 2-grams** and **character 3-grams** were generated. Additionally, **word-level 2-grams** were created using consecutive pairs of words.

Each document was represented as a **set of k-grams**, ensuring that duplicate k-grams were removed. Using these sets, **Jaccard similarity** was computed between all possible document pairs. Jaccard similarity measures the overlap between two sets and is defined as the size of the intersection divided by the size of the union of the sets.

This experiment demonstrates how different representations of text (character vs word n-grams) affect similarity measurements between documents.

```
[7]: for name in char3:
         print(f"{name} - Number of 3-grams:", len(char3[name]))

     D1 - Number of 3-grams: 765
     D2 - Number of 3-grams: 762
     D3 - Number of 3-grams: 828
     D4 - Number of 3-grams: 698
```

```
for name in word2:
    print(f"{name} - Number of word 2-grams:", len(word2[name]))

D1 - Number of word 2-grams: 279
D2 - Number of word 2-grams: 278
D3 - Number of word 2-grams: 337
D4 - Number of word 2-grams: 232
```

```
for name in char2:
    print(f"{name} - Number of 2-grams:", len(char2[name]))

D1 - Number of 2-grams: 263
D2 - Number of 2-grams: 262
D3 - Number of 2-grams: 269
D4 - Number of 2-grams: 255
```

Figure 1: Number of generated character and word k-grams for each document

```
Character 2-gram Similarities
D1-D2: 0.9811320754716981
D1-D3: 0.8156996587030717
D1-D4: 0.6444444444444445
D2-D3: 0.8
D2-D4: 0.6412698412698413
D3-D4: 0.6529968454258676

Character 3-gram Similarities
D1-D2: 0.977979274611399
D1-D3: 0.5803571428571429
D1-D4: 0.3050847457627119
D2-D3: 0.5680473372781065
D2-D4: 0.30590339892665475
D3-D4: 0.31212381771281167

Word 2-gram Similarities
D1-D2: 0.9407665505226481
D1-D3: 0.18234165067178504
D1-D4: 0.03024193548387097
D2-D3: 0.1736641221374046
D2-D4: 0.030303030303030304
D3-D4: 0.01607142857142857
```

Figure 2: Jaccard similarity between document pairs using different k-gram representations.

The results show that **documents D1 and D2 have the highest similarity**, indicating that they contain very similar content. In contrast, **D4 shows very low similarity with other documents**, suggesting that its content is significantly different. Character-level k-grams capture more fine-grained similarity, while word-level k-grams show stricter similarity measurements.

2. MinHash Similarity Estimation

Implement the MinHash algorithm using character 3-grams to estimate document similarity and compare the approximate similarity with the exact Jaccard similarity for different numbers of hash functions.

Answer:

In this section, **MinHashing** was used to estimate the Jaccard similarity between documents efficiently.

While exact Jaccard similarity requires comparing all elements in the sets, MinHash provides a **compact signature representation** that approximates similarity using hash functions.

For each document, a set of **character 3-grams** was generated. These sets were then processed using multiple hash functions to create MinHash signatures. The similarity between two documents was estimated by comparing the fraction of matching hash values in their signatures.

Experiments were performed with different numbers of hash functions (t = 20, 60, 150, 300, 600). Increasing the number of hash functions improves the accuracy of the similarity estimation.

[21]:

```
exact_sim = jaccard(char3["D1"], char3["D2"])
print("Exact Jaccard Similarity (3-grams):", exact_sim)
```

```
Exact Jaccard Similarity (3-grams): 0.977979274611399
```

Figure 3: Exact Jaccard similarity between documents D1 and D2 using character 3-grams.

```
    approx_sim = signature_similarity(sig1, sig2)

    print(f"t = {t} → Approx Similarity: {approx_sim}")
t = 20 → Approx Similarity: 0.95
t = 60 → Approx Similarity: 0.9666666666666667
t = 150 → Approx Similarity: 0.96
t = 300 → Approx Similarity: 0.9766666666666667
t = 600 → Approx Similarity: 0.9833333333333333
```

Figure 4: Approximate similarity between documents D1 and D2 using MinHash signatures with different numbers of hash functions.

The results show that as the number of hash functions increases, the MinHash estimate becomes closer to the exact Jaccard similarity. With larger signature sizes, the estimation error decreases and the similarity approximation becomes more reliable.

3. Locality Sensitive Hashing (LSH):

Apply Locality Sensitive Hashing (LSH) on MinHash signatures to identify candidate document pairs with high similarity and compute the probability of detecting each pair.

Answer:

In this section, **Locality Sensitive Hashing (LSH)** was implemented to efficiently identify document pairs with high similarity. While MinHash signatures approximate similarity, LSH further reduces computation by grouping similar signatures into the same buckets using the **banding technique**.

The MinHash signature matrix was divided into **bands (b)** and **rows per band (r)** such that b×r=tb \times r = tb×r=t, where *t* is the number of hash functions. If two documents share identical hash values in at least one band, they are considered **candidate pairs**.

Using this technique, document pairs with similarity above the threshold (τ = 0.7) were detected efficiently. The probability of detection was also computed using the **LSH S-curve probability formula**:

P(s) = 1 - (1 - s^r)^b

Where S is the Jaccard Similarity.

```
r=5, b=32 → approx threshold ≈ 0.5
r=8, b=20 → approx threshold ≈ 0.6876560219336321
r=10, b=16 → approx threshold ≈ 0.757858283255199
r=16, b=10 → approx threshold ≈ 0.8659643233600653
r=20, b=8 → approx threshold ≈ 0.9012504626108302
r=32, b=5 → approx threshold ≈ 0.9509489152433014
r=40, b=4 → approx threshold ≈ 0.9659363289248456
```

Figure 5: Approximate threshold estimation for different band and row configurations.

```python
candidates = lsh_candidate_pairs(signatures, r, b)

print("Candidate pairs found by LSH:")
print(candidates)
```

```
Candidate pairs found by LSH:
{('D1', 'D2'), ('D3', 'D4')}
```

Figure 6: Candidate document pairs detected using LSH banding.

```
LSH Probability for each pair:

D1-D2: similarity=0.9780, probability=1.000000
D1-D3: similarity=0.5804, probability=0.228224
D1-D4: similarity=0.3051, probability=0.001500
D2-D3: similarity=0.5680, probability=0.195880
D2-D4: similarity=0.3059, probability=0.001532
D3-D4: similarity=0.3121, probability=0.001800
```

Figure 7: LSH detection probability for different document pairs.

The results show that document pair **D1–D2** has a very high probability of being detected due to its high similarity. Other pairs have much lower detection probabilities, indicating lower similarity. This demonstrates how LSH effectively filters highly similar pairs while reducing the number of comparisons.

Question 4 — MinHash on MovieLens Dataset.

Use the MovieLens 100k dataset to compute user similarity based on rated movies and evaluate MinHash approximation by measuring false positives and false negatives.

Answer:

In this section, MinHash was applied to a real-world dataset, the **MovieLens 100k dataset**, to compute similarity between users based on the movies they rated.

Each user was represented as a **set of movie IDs** corresponding to the movies they rated. Exact Jaccard similarity was first computed for all pairs of users. User pairs with similarity greater than or equal to 0.5 were identified as ground truth similar pairs.

MinHash signatures were then generated using **50, 100, and 200 hash functions**. The similarity between users was estimated from these signatures, and the results were compared with the exact similarity to measure **false positives** and **false negatives**.

```
Number of user pairs with similarity >= 0.5: 10
Time taken (seconds): 2.146466016769409
```

Figure 8: Dataset statistics and number of similar user pairs using exact Jaccard similarity.

```
Average Results for t=50:
Average False Positives: 156.4
Average False Negatives: 2.4

[39]:
avg_fp_100, avg_fn_100 = average_runs(user_movies, exact_pairs, 100)

print("\nAverage Results for t=100:")
print("Average False Positives:", avg_fp_100)
print("Average False Negatives:", avg_fn_100)

Run 1 for t=100
Run 2 for t=100
Run 3 for t=100
Run 4 for t=100
Run 5 for t=100

Average Results for t=100:
Average False Positives: 28.8
Average False Negatives: 2.6

[40]:
avg_fp_200, avg_fn_200 = average_runs(user_movies, exact_pairs, 200)

print("\nAverage Results for t=200:")
print("Average False Positives:", avg_fp_200)
print("Average False Negatives:", avg_fn_200)

Run 1 for t=200
Run 2 for t=200
Run 3 for t=200
Run 4 for t=200
Run 5 for t=200

Average Results for t=200:
Average False Positives: 11.0
Average False Negatives: 1.4
```

Figure 9: Average false positives and false negatives for different MinHash signature sizes.

The results show that increasing the number of hash functions improves the accuracy of MinHash similarity estimation. As the signature size increases from 50 to 200, the number of false positives decreases significantly, demonstrating improved approximation of the true Jaccard similarity.

Number of Hash Functions(t)	Average False Positives	Average False Negatives
50	156.4	2.4
100	28.8	2.6
200	11.0	1.4

Table 1: Performance of MinHash similarity estimation on the MovieLens dataset for different signature sizes.

5. Locality Sensitive Hashing on MovieLens Dataset:

Apply LSH on MinHash signatures of MovieLens users to efficiently detect similar user pairs and analyze performance for different banding configurations and similarity thresholds.

Answer:

In this section, **Locality Sensitive Hashing (LSH)** was applied to the MinHash signatures of users in the MovieLens dataset to efficiently identify similar user pairs.

Each user was represented by a set of movies they rated, and MinHash signatures were generated using different numbers of hash functions. The signatures were then divided into bands and rows according to the LSH banding technique.

Different configurations of bands (*b*) and rows (*r*) were tested to evaluate how LSH performs under different parameter settings. The experiments were conducted for two similarity thresholds: **τ = 0.6** and **τ = 0.8**.

For each configuration, the candidate pairs detected by LSH were compared with the exact similar pairs obtained from Jaccard similarity. The results were evaluated using **false positives** and **false negatives**, averaged over multiple runs.

```
LSH Results (t=50, r=5, b=10, threshold=0.6)
Average False Positives: 667.0
Average False Negatives: 0.8
```

```
LSH Results (t=100, r=5, b=20, threshold=0.6)
Average False Positives: 1439.2
Average False Negatives: 0.2
```

```
LSH Results (t=200, r=5, b=40, threshold=0.6)
Average False Positives: 2189.2
Average False Negatives: 0.0
```

```
LSH Results (t=200, r=10, b=20, threshold=0.6)
Average False Positives: 4.4
Average False Negatives: 1.0
```

Figure 10: LSH performance for similarity threshold τ = 0.6 using different banding configurations.

```
LSH Results (t=50, r=5, b=10, threshold=0.8)
Average False Positives: 553.0
Average False Negatives: 0.0
```

```
LSH Results (t=100, r=5, b=20, threshold=0.8)
Average False Positives: 1214.8
Average False Negatives: 0.0
```

```
LSH Results (t=200, r=5, b=40, threshold=0.8)
Average False Positives: 2429.6
Average False Negatives: 0.0
```

```
LSH Results (t=200, r=10, b=20, threshold=0.8)
Average False Positives: 3.0
Average False Negatives: 0.0
```

Figure 11: LSH performance for similarity threshold τ = 0.8 using different banding configurations.

The results show that LSH performance strongly depends on the choice of banding parameters. Configurations with smaller row sizes tend to produce more candidate pairs, leading to higher false positives. Increasing the number of rows per band improves precision by reducing false positives.

For both thresholds, the configuration **t = 200, r = 10, b = 20** provided the best balance between precision and recall, achieving very low false positives while still detecting highly similar user pairs.

6. Conclusion

This assignment demonstrated the use of MinHash and Locality Sensitive Hashing for efficient similarity detection. K-gram representations and Jaccard similarity were used to

measure document similarity. MinHash provided a scalable approximation method, and increasing the number of hash functions improved accuracy. LSH further reduced computation by identifying candidate pairs without comparing all possible pairs. Experiments on the MovieLens dataset showed how parameter choices affect the trade-off between false positives and false negatives.